

PhishLens - Project Report Writing Artifact

Explainable Ensemble Machine Learning (EEML) for Phishing URL Detection

Explainable Ensemble Machine Learning (EEML) for Phishing URL Detection

Use this document as a guide and draft bank when writing your Master's dissertation / project report. Copy, adapt, and cite carefully. Do not paste blindly without checking figures against your latest run.

Field	Value
Last aligned	2026-07-22 (approx.)
Primary artefact	Streamlit app (streamlit_app/)
Evidence files	evaluation/metrics.json, trained_mode...

1. One-paragraph project summary (paste-ready)

PhishLens is a functional Explainable Ensemble Machine Learning (EEML) prototype for phishing URL detection, implemented as a local Streamlit web application. Users submit a URL and receive an immediate classification (phishing or legitimate), confidence score, risk level, and a plain-English explanation of why the model reached that decision. The system combines Logistic Regression, Random Forest and Support Vector Machine (SVM) in a soft-voting ensemble, and uses SHAP (SHapley Additive Explanations) to show which URL features influenced each prediction. The artefact demonstrates how ensemble learning and Explainable AI (XAI) can support transparent cybersecurity decision-making in a practical, inspectable research prototype.

2. Research question & objectives (paste-ready)

Research question:

Can an Explainable Ensemble Machine Learning model improve phishing URL detection while providing transparent SHAP-based explanations that support analyst interpretation and user confidence?

Objectives:

1. Design and implement a Streamlit-based phishing URL detection prototype.
 2. Train a soft-voting ensemble of Logistic Regression, Random Forest and SVM.
 3. Integrate SHAP to produce visual and plain-English explanations per URL.
 4. Evaluate and compare individual models against the ensemble using accuracy, precision, recall, F1-score and ROC-AUC on a held-out test set.
 5. Provide a confidence-evidence mechanism to capture whether explanations improve user confidence after viewing SHAP narratives.
-

3. Proposed artefact (Section 2.4 style - aligned to implementation)

The proposed artefact for this research is a functional Explainable Ensemble Machine Learning (EEML) phishing detection system implemented as a Streamlit-based web application prototype. The system enables users to input a URL and receive an immediate classification indicating whether the URL is legitimate or

phishing.

The artefact combines Logistic Regression, Random Forest and Support Vector Machine using a soft-voting ensemble classifier to improve prediction robustness through complementary inductive biases (linear, tree-based and margin-based learners). To increase transparency, SHAP is integrated to provide visual explanations and human-readable narratives showing which URL features influenced each prediction.

The prototype is implemented in Python. Model development is supported via a reproducible offline pipeline and a Google Colab-compatible notebook workflow (notebooks/phishlens_colab.ipynb), with Streamlit providing the interactive user interface. Although the system runs locally for evaluation and demonstration, its architecture shows how explainable machine learning can be applied in practical phishing detection and cybersecurity decision support.

The artefact has been informed by academic research on phishing detection, ensemble learning and Explainable Artificial Intelligence (XAI), while its design reflects practical requirements for usability, transparency and decision support.

4. Expected / delivered outcomes (Section 2.5 style)

The project delivers the successful development and evaluation of a functional explainable phishing detection prototype. Specifically:

Status	Outcome	Evidence
DELIVERED	Working Streamlit phishing detection ...	streamlit_app/app.py and pages
DELIVERED	Trained ensemble classifier (LR + RF ...	trained_models/ensemble.joblib
DELIVERED	SHAP visualisations + plain-English n...	evaluation/shap/; Explainability & an...
DELIVERED	Performance comparison: individuals v...	evaluation/metrics.json -> model_comp...
DELIVERED	Acc / Prec / Rec / F1 / ROC-AUC on he...	evaluation/metrics.json; evaluation/p...
DELIVERED*	XAI transparency / user-confidence ev...	Confidence Evidence page -> evaluatio...

* Collect a small participant sample and report mean confidence uplift in the results chapter.

5. System overview (methodology / design)

5.1 Architecture (two modes)

Offline pipeline (scripts/run_pipeline.py):

Download data -> preprocess/split -> extract features & train -> evaluate -> generate SHAP artifacts -> dataset statistics.

Interactive dashboard (streamlit run streamlit_app/app.py):

User pastes URL -> validate/normalize -> extract 34 lexical/host features -> ensemble predict_proba -> risk mapping -> SHAP explain -> plain-English narrative -> display.

5.2 Application pages

Page	Purpose
------	---------

Landing (app.py)	Project intro + quick URL analyse entry
Dashboard	Latest analysis + ?Why we said Phishi...
URL Detection	Primary classification workflow
Explainability	SHAP charts + plain-English tab
Model Performance	Metrics, confusion matrix, ROC, PR, c...
Dataset Analytics	Corpus balance / sources
Research	Methodology framing for examiners
Confidence Evidence	Pre/post explanation confidence capture

5.3 Ensemble design

Soft-voting VotingClassifier over:

- Logistic Regression - scaled features, class-balanced
- Random Forest - bagged trees
- SVM (RBF) - scaled features, probability enabled

Rationale (rewrite in your own words): Logistic Regression provides an interpretable linear baseline; Random Forest captures non-linear interactions and is robust to noise; SVM models complex decision boundaries. Soft voting averages class probabilities, reducing reliance on any single inductive bias.

5.4 Features (34 lexical / host-based signals)

Examples: URL/hostname/path length, dots/hyphens/digits, subdomain count, HTTPS/HTTP, IP-as-host, suspicious keywords, suspicious TLD, entropy measures, brand-impersonation heuristic.

Important claim: Features are derived from the URL string only - the system does not visit the page, render HTML, or scan email headers.

5.5 Explainability approach

SHAP TreeExplainer is applied to the Random Forest estimator inside the voting ensemble as a computationally tractable proxy for local attributions. The UI translates top SHAP contributors into plain-English warning signs and reassuring signs, plus practical advice (e.g. do not enter passwords).

Examiner note (honest): Explaining a proxy tree model is a known trade-off versus explaining the full voting function. Discuss this as a limitation and methodological choice.

6. Data & evaluation (results chapter)

6.1 Dataset snapshot (evaluation/dataset_stats.json)

Metric	Value
Total URLs	518
Phishing	300 (OpenPhish)
Legitimate	218 (benign list / fallback)
Features	34
Split	60% / 20% / 20% train/val/test (strat...

Random state	42
Approx. sizes	train 310 / val 104 / test 104

Caveat: Some configured public mirrors returned HTTP 404 during download; the prototype therefore uses a smaller corpus than originally intended. External validation on larger datasets is recommended.

6.2 Latest held-out TEST metrics (evaluation/metrics.json)

Ensemble

Metric	Value
Accuracy	1.000 (100%)
Precision	1.000
Recall	1.000
F1	1.000
ROC-AUC	1.000
Test size	104
Confusion	TN=44, FP=0, FN=0, TP=60

Per-model comparison (test)

Model	Accuracy	F1
Logistic Regression	0.971	0.974
Random Forest	1.000	1.000
SVM	1.000	1.000
Ensemble	1.000	1.000

Validation F1 (training metadata)

Model	Val F1
Logistic Regression	0.976
Random Forest	0.984
SVM	0.976
Ensemble	0.976

Latency (approx., 100-sample probe): mean ~55 ms, p50 ~51 ms, p95 ~73 ms.

6.3 How to talk about ?perfect? scores (critical for examiners)

Do not claim the system is production-ready or universally perfect. Prefer:

On the held-out test split of this prototype corpus, the ensemble achieved perfect classification metrics. These results are encouraging for the dissertation demonstration but must be interpreted cautiously: the dataset is relatively small, source diversity is limited, and high scores may partly reflect separable source patterns rather than broad generalisation to unseen phishing campaigns. Future work should validate on larger public benchmarks.

6.4 Screenshots / figures to include

- Dashboard / URL Analysis / Explainability screenshots (data/images/)
- evaluation/plots/confusion_matrix.png

- evaluation/plots/roc_curve.png
 - evaluation/plots/pr_curve.png
 - evaluation/plots/model_comparison.png
 - evaluation/shap/summary_plot.png
 - evaluation/shap/importance_bar.png
 - UI: ?Why we said Phishing/Legitimate? plain-English panel
 - Confidence Evidence page (with sample ratings if collected)
-

7. What makes this artefact distinct (discussion / contribution)

PhishLens is not positioned as a replacement for commercial blockers (e.g. Safe Browsing, VirusTotal). Its contribution is research-oriented:

1. Transparency - explains *why* a URL is phishing/legitimate, not only the label.
2. Ensemble design - LR + RF + SVM with soft voting and explicit comparison.
3. XAI integration - SHAP visuals + non-technical narratives + advice.
4. Local inspectable prototype - full reproducible pipeline and dashboard.
5. Decision-support focus - supports analyst/user confidence, not just blocking.
6. Academic integrity - metrics and SHAP values come from real evaluation artifacts (no fabricated results).

One-line contribution statement:

This project contributes a reproducible EEML phishing-URL prototype that pairs ensemble classification with SHAP-based, human-readable explanations for transparent cybersecurity decision support.

8. Limitations (paste-ready)

- URL-only analysis cannot detect phishing that depends on page content, visual spoofing, or post-click behaviour.
 - Corpus size (~518 URLs) is small relative to production phishing studies.
 - Perfect test metrics warrant cautious interpretation and external validation.
 - Some planned dataset mirrors failed (404), reducing source diversity.
 - SHAP attributions explain the model (via RF proxy), not ground-truth causality.
 - Ensemble-level SHAP is approximated via TreeExplainer on Random Forest.
 - Concept drift: phishing campaigns change; models need periodic retraining.
 - Local research prototype: not hardened for production rate-limiting, adversarial robustness, or enterprise deployment.
 - User-confidence evidence requires collecting participant ratings on the Confidence Evidence page; the mechanism is built, but sample size depends on the study you run.
-

9. Future work (paste-ready)

- Expand and diversify datasets (restore PhiUSIIL / additional corpora).
- External validation on independent public benchmarks.
- Fuse URL features with DOM / screenshot embeddings.
- Analyst-in-the-loop feedback for active learning.

- Adversarial robustness tests (URL perturbation / homoglyphs).
- Formal user study measuring confidence/trust uplift with/without SHAP.
- Explore KernelExplainer or ensemble-level attribution methods.

10. How to run / reproduce (appendix)

One-shot:

```
python scripts/setup_and_run.py
```

Manual:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python scripts/run_pipeline.py
streamlit run streamlit_app/app.py
```

Tests:

```
pytest tests/ -v
```

Colab-oriented notebook: notebooks/phishlens_colab.ipynb

11. Requirements traceability (2.4 / 2.5 checklist)

Requirement	Status	Evidence
Streamlit URL phishing classifier	PASS	streamlit_app/
Ensemble of LR + RF + SVM	PASS	training/train.py, trained_models/*
SHAP visual explanations	PASS	training/generate_shap.py, pages/3_Ex...
Plain-English explainability for non-...	PASS	services/explainer.py, analysis_panel...
Model comparison vs ensemble	PASS	evaluate.py, pages/4_Model_Evaluation.py
Metrics: Acc / Prec / Rec / F1 / ROC-AUC	PASS	evaluation/metrics.json
Colab-compatible development path	PASS	notebooks/phishlens_colab.ipynb
XAI transparency / confidence evidenc...	PASS*	pages/7_Confidence_Evidence.py (*fill...
Python local prototype demonstrating ...	PASS	README + app

12. Suggested report chapter map

Chapter / Section	Use content from
Introduction / Problem	§2, §7
Proposed Artefact (2.4)	§3
Expected Outcomes (2.5)	§4
Literature / Related work	Add your own citations (SHAP, RF, SVM...
Design / Architecture	§5

Implementation	§5 + project structure in README
Evaluation / Results	§6 (+ screenshots)
Discussion / Contribution	§7
Limitations	§8
Future Work	§9
Conclusion	Combine §1 + §7 carefully (no overclaim)
Appendix / Reproduction	§10
Traceability (optional appendix)	§11

13. Academic integrity statement (paste-ready)

All metrics, plots and SHAP values presented in the PhishLens Streamlit application are computed from real model outputs on the processed dataset splits. No fabricated evaluation results are displayed. Dataset provenance and preprocessing methodology are documented in data/DATASET.md. The React UI under src/ is a design reference only and is not the dissertation artefact.

14. Glossary (optional appendix)

Term	Meaning
Phishing	Deceptive links aiming to steal crede...
Ensemble	Multiple models combined; here soft v...
SHAP	Method to attribute each feature?s co...
Soft voting	Average predicted class probabilities...
Lexical features	Properties from the URL text (not fro...
XAI	Explainable Artificial Intelligence
EEML	Explainable Ensemble Machine Learning
Stratified split	Train/val/test split preserving class...

End of artifact - PhishLens report writing pack